

Lors de la première semaine j’ai étudié en Profondeur Openstack, la documentation étant très centrée sur l’utilisation , et non sur la terminologie de chacun des composants je devais m’imprégner du rôle de chaque composant , de leur utilité, de comment il fonctionne , je devais aussi et surtout identifier ceux qui seraient les plus susceptibles d’être utilisés. Pour cela j’ai lu (et relu) le livre intitulé Openstack Essentials, il est dans la documentation à situer dans le répertoire Documentation, j’ai aussi fait un petit résumé que vous trouverez à la suite de ce message, ça peut faciliter l’appréhension de beaucoup de termes qui seront utilisés dans tous les autres fichiers de code ou de documentation, c’est vraiment la base pour se familiariser avec Openstack et son environnement .

Aussi au début , le stage était fait pour travailler sur les machines virtuelles de virtualbox , donc il se peut que certaines informations présentées par la suite soit implémentable sur cette dernière, alors que notre environnement c’est GRID5000, dans la prochaine documentation(suivant la Roadmap) on ajoutera les spécificités liés à cet environnement et comment adapter les commandes

NB: Bien regarder NOVA , GLANCE, NEUTRON, CINDER (ils seront très important dans la suite)

I. Chapitre 1 : Les composants d'Openstack

Résumé détaillé du Chapitre 1 : Vue d'ensemble de l'architecture et des composants d'OpenStack

Le chapitre 1 de *OpenStack Essentials* présente l'architecture d'OpenStack, une plateforme open source de cloud computing permettant de virtualiser les ressources de calcul, de réseau et de stockage. OpenStack est modulaire, offrant une grande flexibilité pour adapter les déploiements aux besoins spécifiques.

A)Architecture d'OpenStack

L'architecture se divise en trois couches principales :

1. Couche de contrôle (Control Tier)

- Gère les interfaces API, la base de données et le bus de messages.
- Elle assure la coordination des différents services.

2. Couche réseau (Network Tier)

- Gère la virtualisation des réseaux à l'aide de technologies comme le SDN (Software Defined Networking).

3. Couche de calcul (Compute Tier)

- Responsable de l'exécution des machines virtuelles (VM) à travers des hyperviseurs.

B) Composants clés d'OpenStack

1. Dashboard (Horizon)

- Interface utilisateur web pour gérer et surveiller les ressources OpenStack.
- Permet d'interagir avec les autres composants sans utiliser la ligne de commande.

2. Keystone (Gestion des identités)

- Service d'authentification et de gestion des identités.
- Il gère les utilisateurs, les rôles, et les projets (ou "tenants").
- Centralise les informations sur les points d'accès API des autres services, facilitant la communication entre eux.

3. Glance (Gestion des images)

- Service de gestion des images disque utilisées pour lancer des instances.
- Les images sont des systèmes d'exploitation préinstallés sans configurations spécifiques (images "scellées").
- Glance optimise le lancement d'instances en mettant en cache les images sur les nœuds de calcul.

4. Neutron (Gestion réseau)

- Fournit des fonctionnalités avancées de virtualisation réseau (NaaS).
- Permet aux utilisateurs de définir des réseaux virtuels isolés, des routeurs virtuels et des passerelles externes.
- Supporte l'intégration de plugins pour une configuration réseau flexible (par exemple Open vSwitch).

5. Nova (Gestion des instances)

- Composant central pour le lancement et la gestion des machines virtuelles.
- Planifie l'exécution des instances sur les nœuds de calcul en fonction des ressources disponibles.
- Assure la connexion réseau et la configuration de clés SSH.

6. Cinder (Stockage bloc)

- Offre une gestion des volumes de stockage de type bloc.
- Les volumes peuvent être attachés aux instances et utilisés comme des disques classiques.
- Supporte les instantanés et plusieurs backends de stockage (LVM, Ceph, etc.).

7. Swift (Stockage objet)

- Service de stockage d'objets simple et évolutif.
- Stocke des fichiers sous forme de contenu brut sans métadonnées de système de fichiers.
- Permet la réplication des données pour assurer la haute disponibilité.

8. Ceilometer (Télémétrie)

- Collecte des données sur l'utilisation des ressources pour la facturation et la surveillance.
- Mesure l'utilisation des ressources sous forme d'échantillons (samples) qui peuvent générer des statistiques et des alertes.

9. Heat (Orchestration)

- Automatisation du déploiement et de la configuration des ressources.
- Utilise des fichiers de modèles (templates) pour définir les ressources et leurs dépendances.
- Permet de coordonner le lancement de multiples instances, souvent en combinaison avec des outils de gestion de configuration.

C) Modularité et évolutivité

- L'architecture modulaire permet de sélectionner uniquement les composants nécessaires.
- OpenStack peut être installé sur une seule machine pour des tests ou étendu horizontalement à plusieurs nœuds pour des déploiements en production.

II. Chapitre 2 : Installation de Packstack incontournable a l'utilisation de OpenStack

Chapitre 2 : Installation de RDO (Packstack)

Introduction

Ce chapitre présente l'installation d'OpenStack via la distribution Debian ou Ubuntu et l'outil d'installation Devstack.

Pour installer **DevStack** sans avoir besoin de VirtualBox, suivez cette procédure simplifiée adaptée à votre système Ubuntu :

Étape 1 : Préparer votre environnement

Assurez-vous que votre système est à jour :

```
sudo apt update && sudo apt upgrade -y
```

Installez les outils nécessaires :

```
sudo apt install git net-tools -y
```

Étape 2 : Cloner DevStack

Clonez le dépôt DevStack :

```
git clone https://opendev.org/openstack/devstack
```

```
cd devstack
```

Étape 3 : Configurer DevStack

Créez un fichier de configuration minimal **local.conf** :

```
nano local.conf
```

Ajoutez la configuration suivante :

```
[[local|localrc]]
```

```
ADMIN_PASSWORD=admin
```

```
DATABASE_PASSWORD=admin
```

RABBIT_PASSWORD=admin

SERVICE_PASSWORD=admin

Enregistrez et quittez (CTRL + X, puis Y).

Étape 4 : Lancer l'installation

Exécutez la commande suivante pour démarrer l'installation d'OpenStack :

```
./stack.sh
```

Remarque importante

- Cela peut prendre du temps en fonction de votre connexion Internet et de votre matériel.
 - Si des erreurs apparaissent, partagez les messages pour que je puisse vous aider à les corriger.
-

Accès à l'interface OpenStack

Une fois l'installation terminée, l'interface web sera disponible à l'adresse indiquée dans le terminal, souvent :

`http://localhost/dashboard`

Connectez-vous avec l'utilisateur `admin` et le mot de passe défini (`admin` dans l'exemple).

Architecture multi-nœuds (Installation avancée)

Pour une configuration plus complexe, il est recommandé d'utiliser une architecture avec trois nœuds distincts :

- **Nœud de contrôle** : Gestion des API et base de données.
- **Nœud réseau** : Configuration SDN via Neutron.
- **Nœud de calcul** : Exécution des machines virtuelles.

L'utilisation de fichiers de configuration (**answer file**) permet de personnaliser les installations multinœuds.

Chapitre 3 : Gestion des identités avec Keystone

Présentation de Keystone

- Composant central pour l'authentification et la gestion des utilisateurs, rôles et projets.
- Gère un catalogue d'API pour faciliter la communication entre les composants.

Hiérarchie des utilisateurs et rôles

- **Projet (Tenant)** : Regroupement de ressources virtuelles.
- **Utilisateur** : Entité qui accède à OpenStack.
- **Rôle** : Définit les autorisations des utilisateurs au sein d'un projet.

Création de ressources dans Keystone

Créer un utilisateur :

```
keystone user-create --name monUtilisateur
```

●

Créer un projet :

```
keystone tenant-create --name monProjet
```

●

Associer un rôle :

```
keystone user-role-add --user monUtilisateur --tenant monProjet --role membre
```

●

Accès via l'interface web (Dashboard)

- Gestion graphique des utilisateurs et projets dans le menu "Admin > Identité".
-

Chapitre 4 : Gestion des images avec Glance

Présentation de Glance

- Service de gestion des images disque utilisées pour lancer des instances.
- Les images doivent être préconfigurées (sans clés SSH ni adresses MAC).

Téléchargement et enregistrement d'images

Importation via la ligne de commande :

```
glance image-create --name "Ubuntu" --disk-format qcow2 --container-format bare  
--file ubuntu.qcow2
```

-

Création d'images personnalisées

- Utilisation d'outils tels que `virt-install` ou `cloud-init`.
-

Chapitre 5 : Gestion réseau avec Neutron

Présentation de Neutron

- Composant responsable de la gestion des réseaux virtuels et de la connectivité.
- Supporte des technologies comme VLAN, VXLAN et tunnels GRE.

Fonctionnalités principales

- **Réseaux isolés** : Chaque projet peut définir ses propres réseaux.
- **Routeurs virtuels** : Connexion entre réseaux virtuels.
- **IP flottantes** : Accès externe aux instances.

Exemple de création d'un réseau

```
neutron net-create monReseau
```

Chapitre 6 : Gestion des instances avec Nova

Présentation de Nova

- Composant principal pour la gestion des machines virtuelles.

Étapes pour lancer une instance

Création d'une paire de clés SSH :

```
nova keypair-add --pub-key ~/.ssh/id_rsa.pub maCle
```

1.

Définition des règles de sécurité :

```
nova secgroup-add-rule default tcp 22 22 0.0.0.0/0
```

2.

Lancement de l'instance :

```
nova boot --flavor m1.small --image "Ubuntu" --key-name maCle monInstance
```

3.

Chapitre 7 : Gestion du stockage bloc avec Cinder

Présentation de Cinder

- Service de gestion des volumes de type bloc.
- Permet l'attachement de volumes aux instances pour augmenter leur capacité de stockage.

Fonctionnalités principales

Création de volumes :

```
cinder create --display-name monVolume 10
```

-

Attachement à une instance :

```
cinder attach monVolume monInstance
```

-
-

Chapitre 8 : Gestion du stockage objet avec Swift

Présentation de Swift

- Stockage d'objets simple et évolutif.
- Contrairement à Cinder, il ne gère pas les métadonnées de type système de fichiers.

Architecture

- **Proxy** : API pour l'interaction avec les utilisateurs.
- **Backend de stockage** : Réplication et distribution des objets.

Commandes clés

Créer un conteneur :

swift post monConteneur

-

Télécharger un fichier :

swift upload monConteneur monFichier.txt

-